

LAPORAN PRAKTIKUM MODUL 4

Identitas Praktikan

Nama : Rajibin Putra Ilman

NIM : 245150407111084

Kelas : D

Mata Kuliah : Pengembangan Aplikasi Mobile

Modul : 4 – State Hoisting, Perubahan Konfigurasi, dan Side-effect

I. Hasil Praktikum

4.5.1 State Hoisting

Aktivitas 1

a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Nilai angka yang ditampilkan pada Composable Text bertambah 1 setiap kali tombol diklik.

b. Apa yang menyebabkan hal tersebut terjadi?

Karena event `onClick` pada `Button` menjalankan kode `number++` sehingga nilai state `number` berubah. Ketika state berubah, Compose melakukan recomposition dan memperbarui tampilan.

c. Variabel state apa yang berperan pada Composable Text?

Variabel `number`.

d. Variabel state apa yang berperan pada label Button?

Variabel `label`.

Aktivitas 2

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Ubah kode pada fungsi `CounterScreen()`:

Sebelum:

Kotlin

```
var number by remember { mutableStateOf(0) }  
  
val label by remember { mutableStateOf("Increment") }  
  
Button(onClick = { number++ })
```

Sesudah:

Kotlin

```
var number by remember { mutableStateOf(0) }  
  
val label by remember { mutableStateOf("Increment 2") }  
  
Button(onClick = { number += 2 })
```

Langkah-langkah

1. Buka MainActivity.kt
2. Cari fungsi CounterScreen()
3. Ubah teks label menjadi "Increment 2"
4. Ubah number++ menjadi number += 2
5. Jalankan aplikasi
6. Ambil screenshot sebelum dan sesudah tombol diklik

Aktivitas 4

a. Apakah program berjalan dengan baik?

Ya. Program tetap berjalan dengan baik dan menghasilkan perilaku yang sama seperti sebelumnya.

b. Di bagian mana proses increment dilakukan?

Pada lambda onClick yang dikirim saat pemanggilan CounterScreen().

Kotlin

```
onButtonClick = { number++ }
```

c. Di mana state Text dan Button dideklarasikan?

State sekarang dideklarasikan di fungsi induk (caller), yaitu pada bagian:

Kotlin

```
var number by remember { mutableStateOf(0) }
```

Sedangkan label dikirim melalui parameter:

Kotlin

```
label = "Increment"
```

Aktivitas 5

a. Label tombol menjadi "Double"

Kotlin

```
label = "Double"
```

b. Nilai awal number menjadi 1

Kotlin

```
var number by remember { mutableStateOf(1) }
```

c. Mengubah fungsi agar menggandakan nilai

Sebelum:

Kotlin

```
onButtonClick = { number++ }
```

Sesudah:

Kotlin

```
onButtonClick = { number *= 2 }
```

d. Kode lengkap

Kotlin

```
var number by remember { mutableStateOf(1) }
```

```
CounterScreen(  
    modifier = Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onButtonClick = {  
        number *= 2  
    }  
)
```

Aktivitas 6

b. Di bagian mana perubahan dilakukan?

Semua perubahan dilakukan pada pemanggilan `CounterScreen()` dan deklarasi state di fungsi induk.

c. Apakah ada perubahan di `CounterScreen()`?

Tidak ada.

`CounterScreen()` tetap sama karena sudah menjadi composable stateless setelah state hoisting dilakukan.

Aktivitas 7

Apa manfaat state hoisting?

State hoisting memudahkan perubahan kebutuhan aplikasi tanpa perlu mengubah composable UI. Logika bisnis dan state dipisahkan dari tampilan sehingga kode lebih mudah dirawat, digunakan kembali, dan diuji.

4.5.2 Perubahan Konfigurasi

Aktivitas 2

b. Berapa angka yang ditampilkan? Mengapa?

Setelah perangkat diputar ke mode landscape, angka kembali ke nilai awal (1).

Hal ini terjadi karena `remember` hanya mempertahankan state selama recomposition dan tidak menyimpan state ketika terjadi perubahan konfigurasi seperti rotasi layar.

Langkah Mengubah `remember` menjadi `rememberSaveable`

1. Buka `MainActivity.kt`
2. Cari kode:

Kotlin

```
var number by remember { mutableStateOf(1) }
```

3.
Ubah menjadi:

Kotlin

```
var number by rememberSaveable { mutableStateOf(1) }
```

4.
Tekan **Alt + Enter**
5. Pilih **Import `rememberSaveable`**
6. Jalankan ulang aplikasi

Aktivitas 3

a. Berapa angka yang ditampilkan?

Angka tetap sama seperti sebelum perangkat diputar.

b. Mengapa?

Karena `rememberSaveable` menyimpan state ke dalam Bundle sehingga nilai tetap dipertahankan saat terjadi perubahan konfigurasi.

c. Fungsi rememberSaveable

`rememberSaveable` digunakan untuk mempertahankan nilai state ketika terjadi perubahan konfigurasi seperti rotasi layar, sedangkan `remember` tidak dapat melakukannya.

Aktivitas 4

Contoh penggunaan `rememberSaveable`:

Form Login.

Kotlin

```
var username by rememberSaveable { mutableStateOf("") }
```

Manfaat:

- Input pengguna tidak hilang saat layar diputar.
- Pengalaman pengguna menjadi lebih baik.
- Tidak perlu mengetik ulang data.

4.5.3 LaunchedEffect

Aktivitas 1

a. Saat pertama kali dijalankan apa yang tampil?

Muncul Snackbar dengan pesan:

None

Number 1 is shown!

(sesuai nilai awal variabel `number`)

b. Mengapa pesan langsung muncul?

Karena `LaunchedEffect(number)` dijalankan saat composable pertama kali masuk ke composition.

c. Apa yang memicu Snackbar?

Perubahan nilai `number`.

Kode yang memicu:

Kotlin

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar(  
        "Number $number is shown!"  
    )  
}
```

Aktivitas 2

a. Apa yang terjadi setelah tombol Double diklik?

- Nilai number berubah.
- Snackbar baru muncul.
- Pesan Snackbar menampilkan nilai terbaru.

b. Mengapa?

Karena perubahan state number menyebabkan `LaunchedEffect(number)` dijalankan ulang.

Pra-Percobaan 2

Langkah-langkah membuat Repo.kt

1. Klik kanan package aplikasi
2. Pilih **New** → **Kotlin Class/File**
3. Nama file: Repo
4. Isi dengan kode:

Kotlin

```
class Repo {  
    companion object {  
        suspend fun getData(): Int {  
            delay(2000)  
            return Random.nextInt(100, 1000)  
        }  
    }  
}
```

```
    }  
  }  
}
```

5.
Import:

```
Kotlin  
import kotlinx.coroutines.delay  
import kotlin.random.Random
```

6.
Tambahkan ke onCreate():

```
Kotlin  
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

Aktivitas 1 (Pra-Percobaan 2)

a. Saat pertama kali aplikasi dijalankan apa yang tampil?

Angka awal yaitu:

```
None  
1
```

b. Setelah 2 detik apa yang tampil?

Angka berubah menjadi angka acak antara 100–999.

Contoh:

None

573

c. Apa yang memicu perubahan tersebut?

`LaunchedEffect(Unit)` menjalankan fungsi:

Kotlin

```
number = Repo.getData()
```

setelah proses simulasi pengambilan data selama 2 detik.

Aktivitas 2

a. Perbedaan `LaunchedEffect(state)` dan `LaunchedEffect(Unit)`

`LaunchedEffect(state)`

Kotlin

```
LaunchedEffect(number)
```

- Dijalankan saat pertama kali composition.
- Dijalankan ulang ketika nilai `number` berubah.

`LaunchedEffect(Unit)`

Kotlin

```
LaunchedEffect(Unit)
```

- Hanya dijalankan satu kali saat composable pertama kali dibuat.
- Tidak dijalankan ulang ketika state berubah.

b. Kapan dijalankan?

`LaunchedEffect(state)`

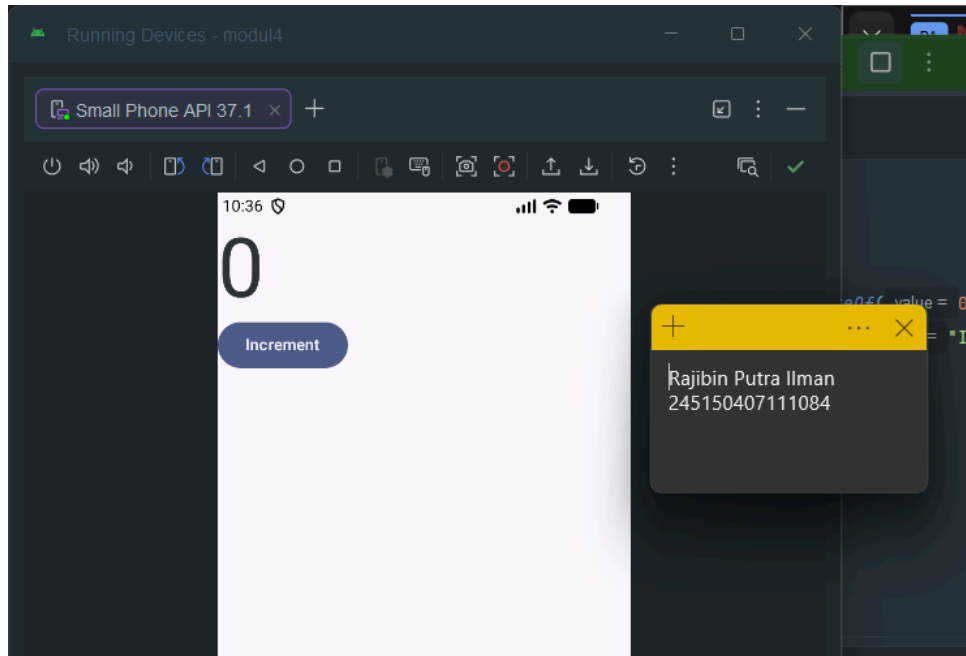
- Saat composable pertama muncul.
- Setiap key/state berubah.

LaunchedEffect(Unit)

- Hanya sekali ketika composible pertama masuk composition.

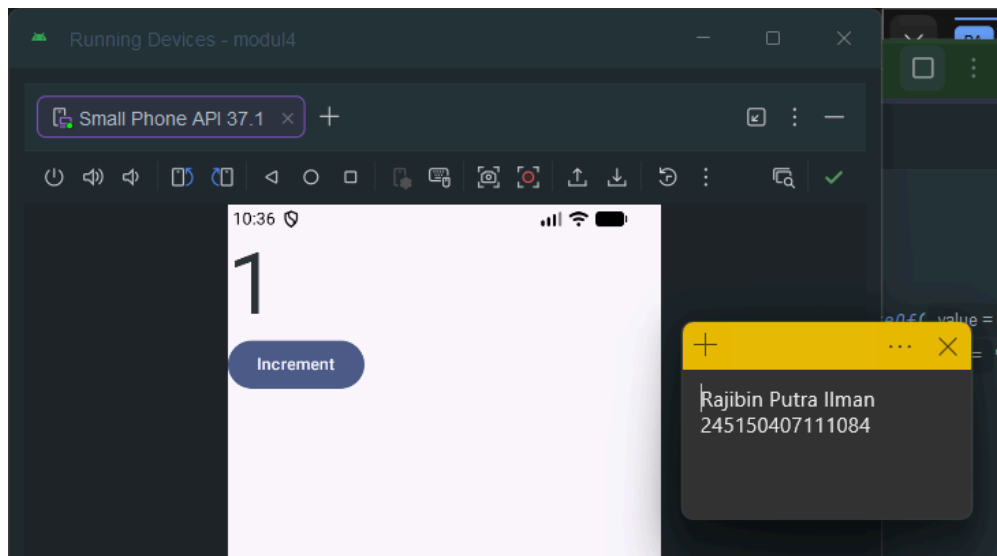
LAMPIRAN

1. Gambar 4.5.1 Aktivitas 1 (Sebelum diklik):



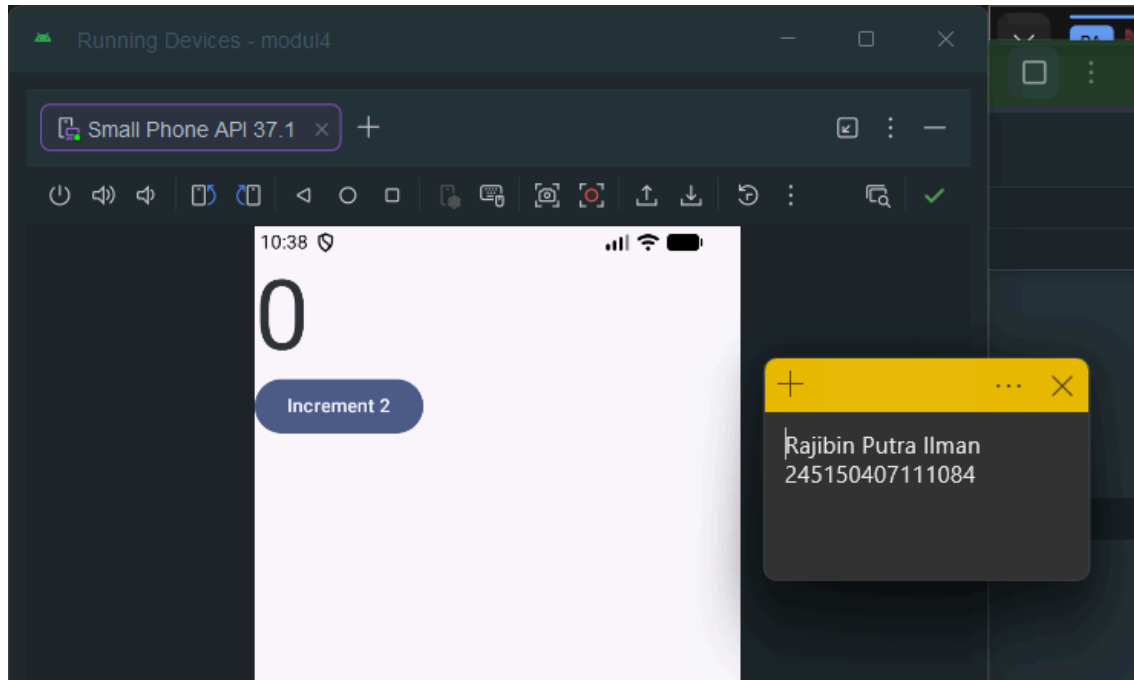
Tampilan awal State Hoisting Aktivitas 1 menunjukkan angka 0 sebelum tombol "Increment" diklik.

2. Gambar 4.5.1 Aktivitas 1 (Setelah diklik):



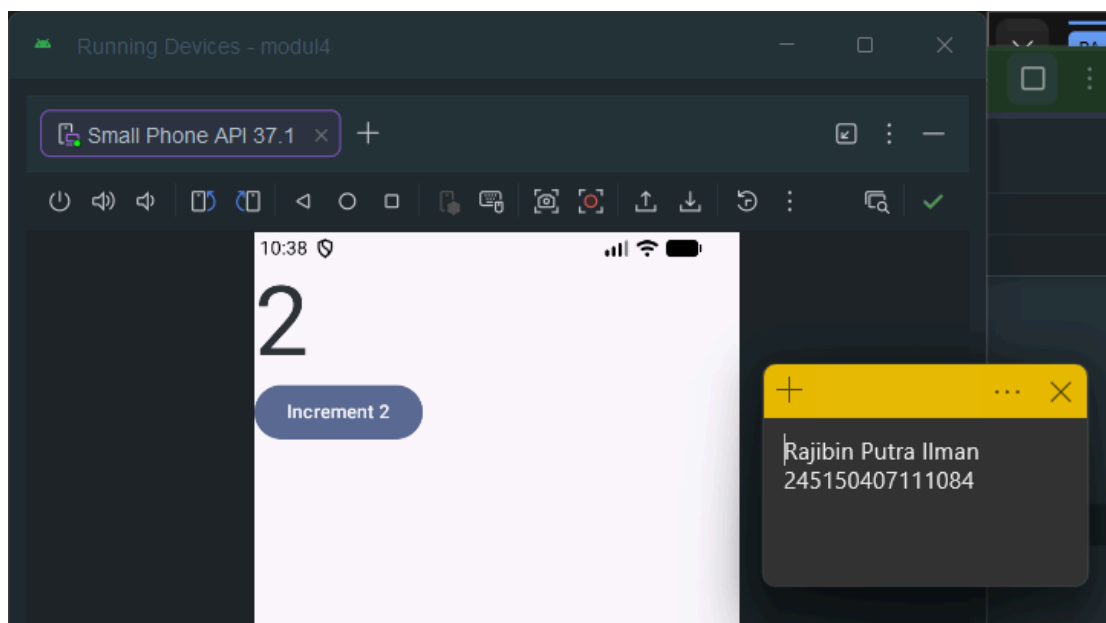
Hasil setelah tombol "Increment" diklik pada Aktivitas 1, di mana nilai angka bertambah menjadi 1.

3. Gambar 4.5.1 Aktivitas 2 (Sebelum diklik):



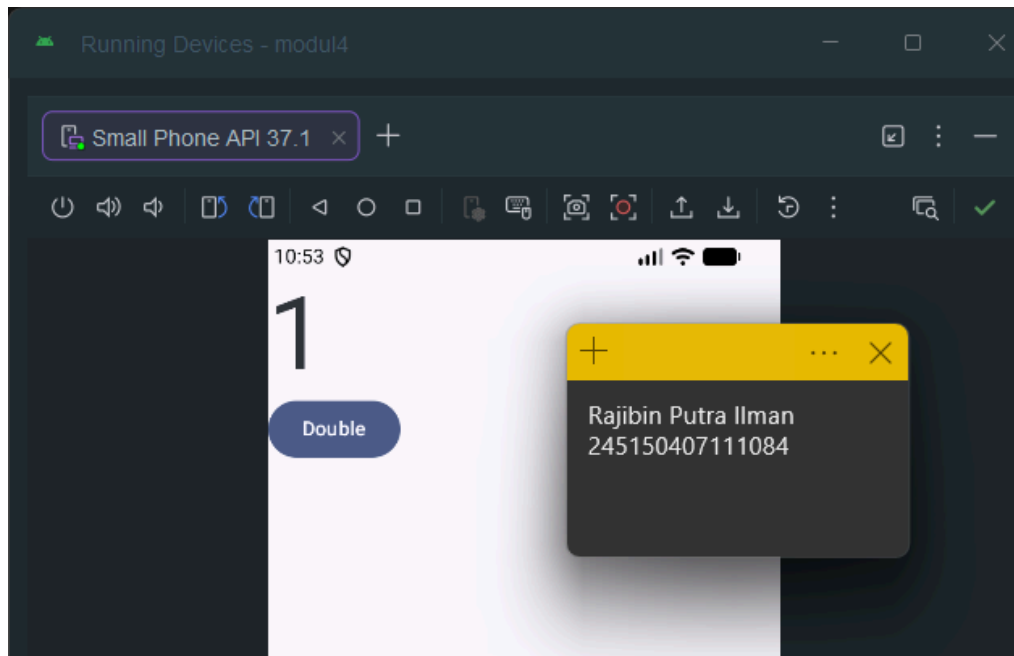
Tampilan awal Aktivitas 2 dengan tombol bertuliskan "Increment 2" dan angka awal 0.

4. Gambar 4.5.1 Aktivitas 2 (Setelah diklik):



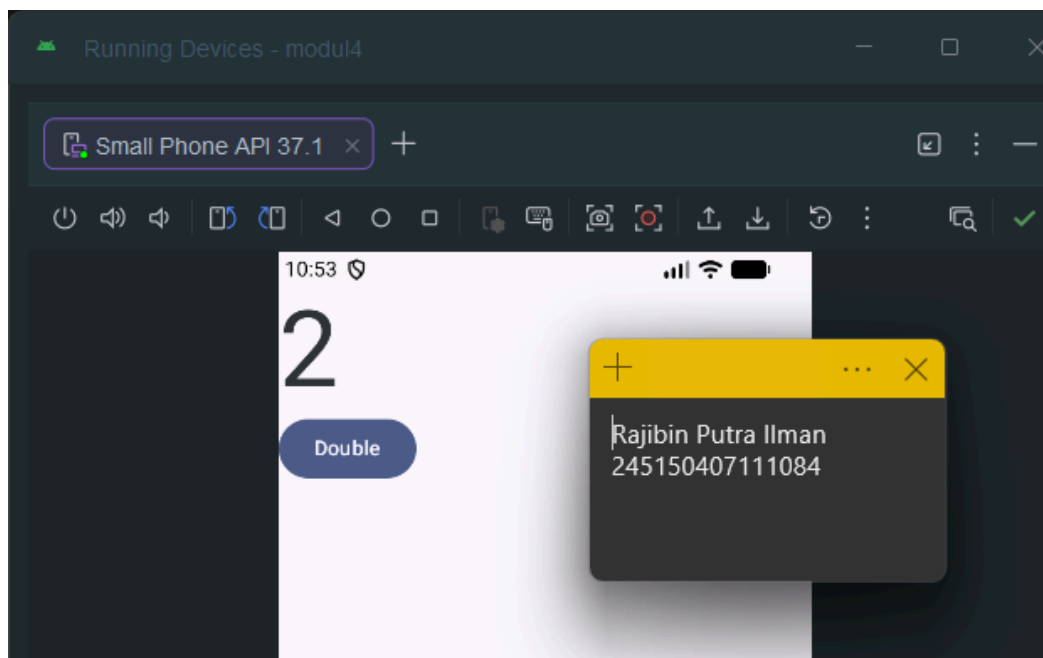
Hasil setelah tombol "Increment 2" diklik, nilai angka bertambah 2 menjadi 2.

5. Gambar 4.5.1 Aktivitas 5 (Nilai Awal):



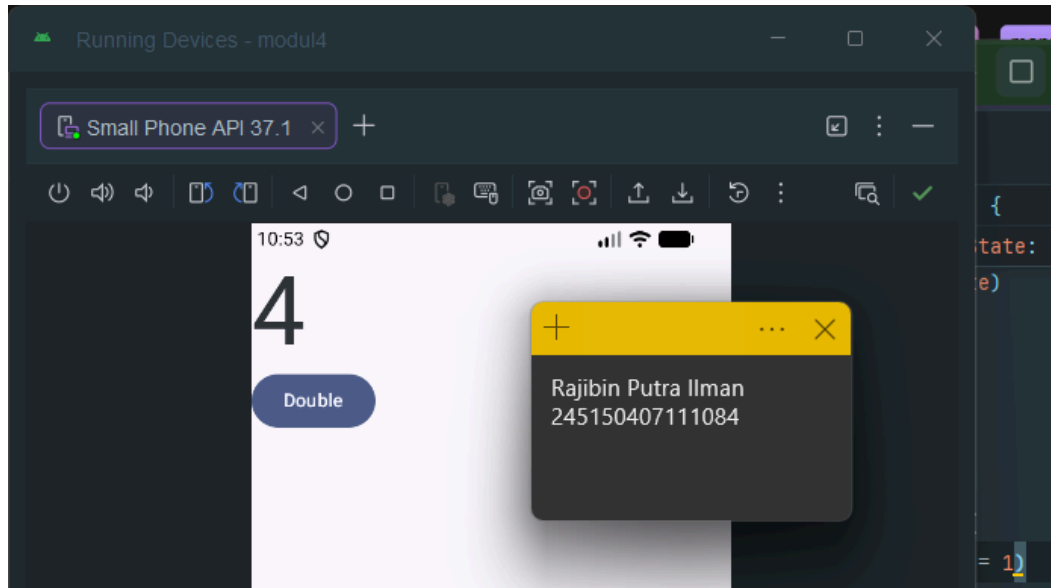
Tampilan awal Aktivitas 5 setelah state hoisting diubah, menampilkan label "Double" dan nilai awal 1.

6. Gambar 4.5.1 Aktivitas 5 (Perkalian Pertama):



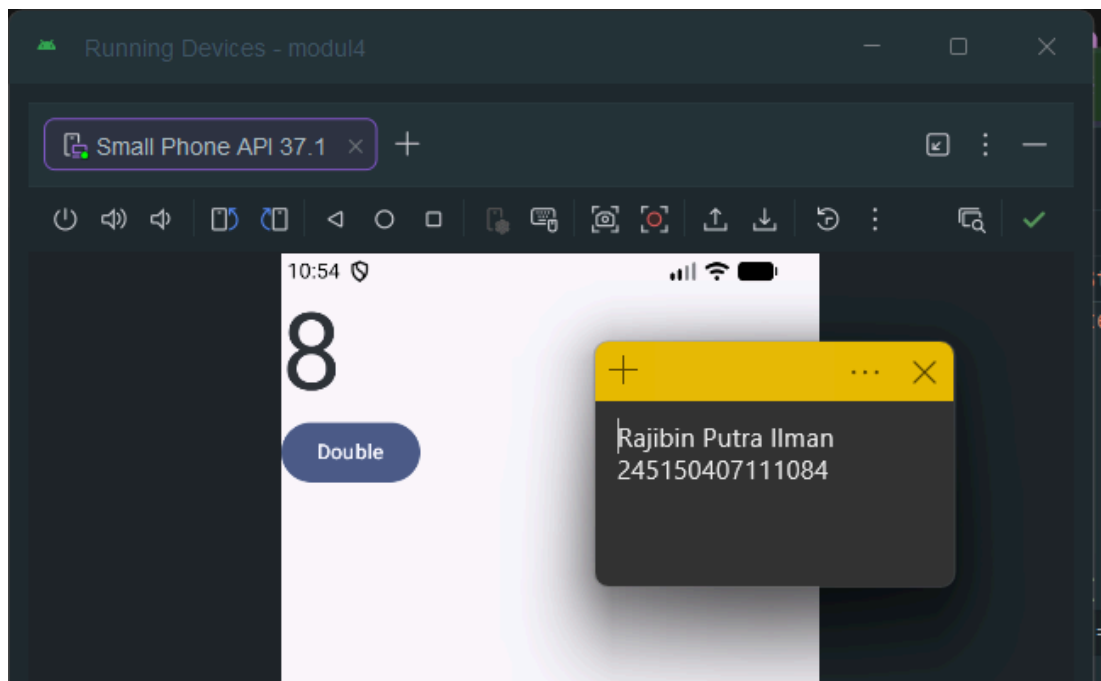
Hasil penggandaan nilai pertama pada Aktivitas 5, angka menjadi 2 setelah tombol "Double" diklik.

7. Gambar 4.5.1 Aktivitas 5 (Perkalian Kedua):



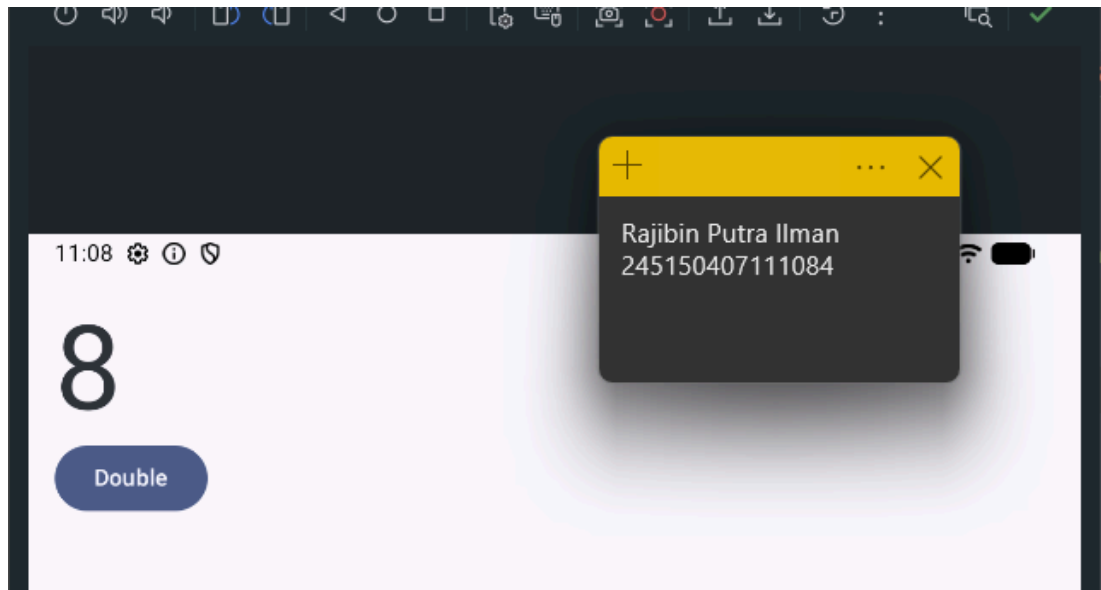
Hasil penggandaan nilai kedua pada Aktivitas 5, angka berubah menjadi 4.

8. Gambar 4.5.1 Aktivitas 5 (Perkalian Ketiga):



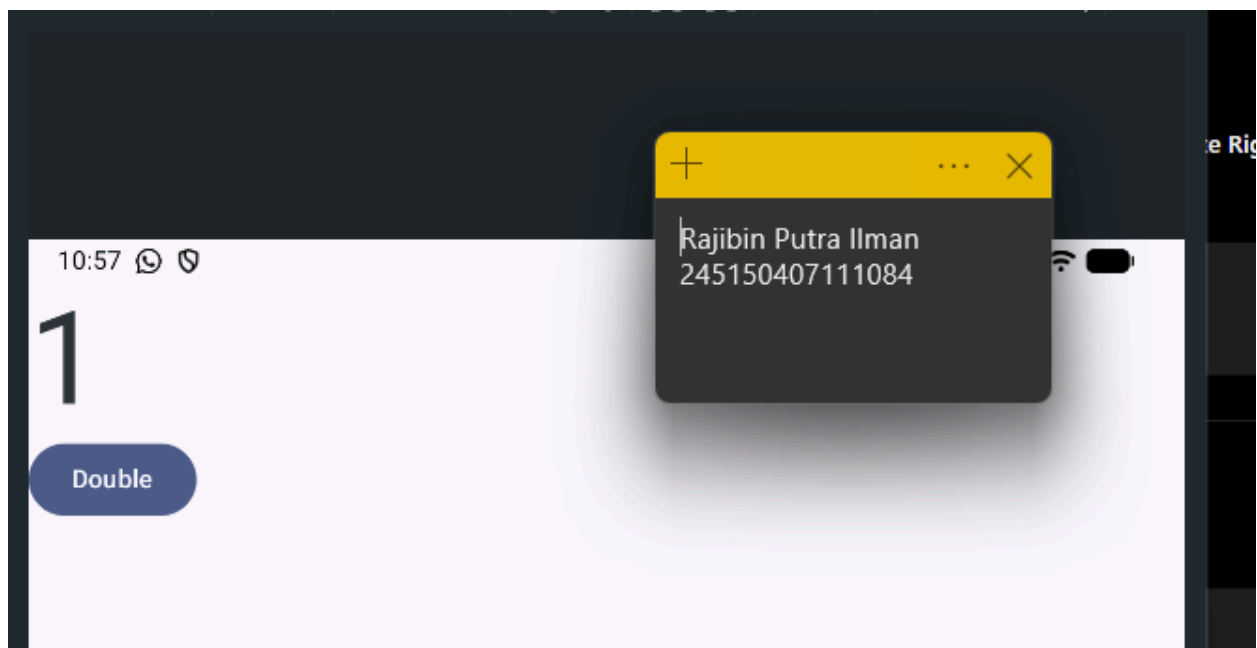
Hasil penggandaan nilai ketiga pada Aktivitas 5, angka bertambah menjadi 8.

9. Gambar 4.5.2 Perubahan Konfigurasi (Sebelum Rotasi):



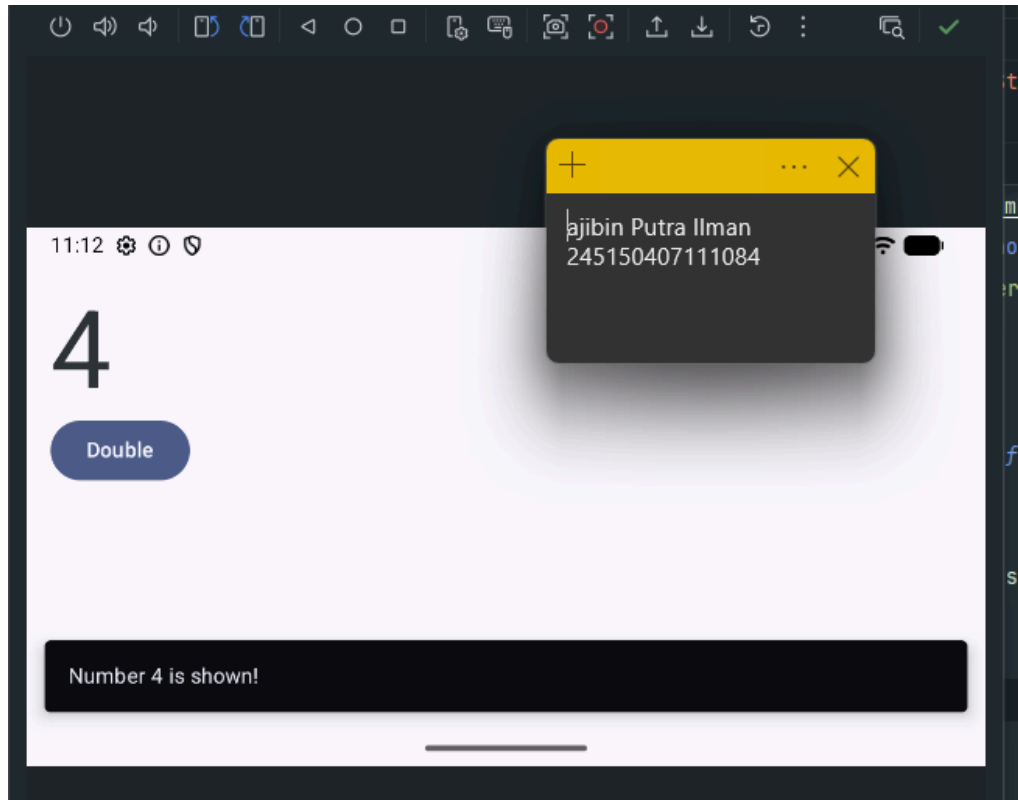
Kondisi nilai state (8) sebelum terjadi perubahan konfigurasi atau rotasi layar.

10. Gambar 4.5.2 Perubahan Konfigurasi (Setelah Rotasi / Reset State):



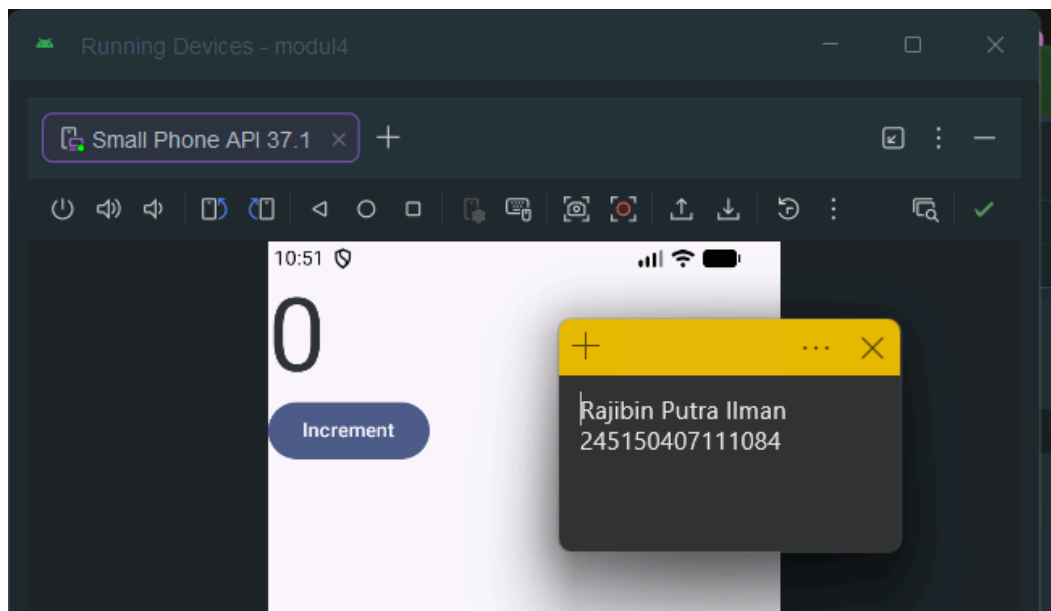
Kondisi ketika menggunakan `remember`, state kembali ke nilai awal (1) setelah perubahan konfigurasi/rotasi.

11. Gambar 4.5.3 LaunchedEffect Percobaan 1 (Snackbar Awal):



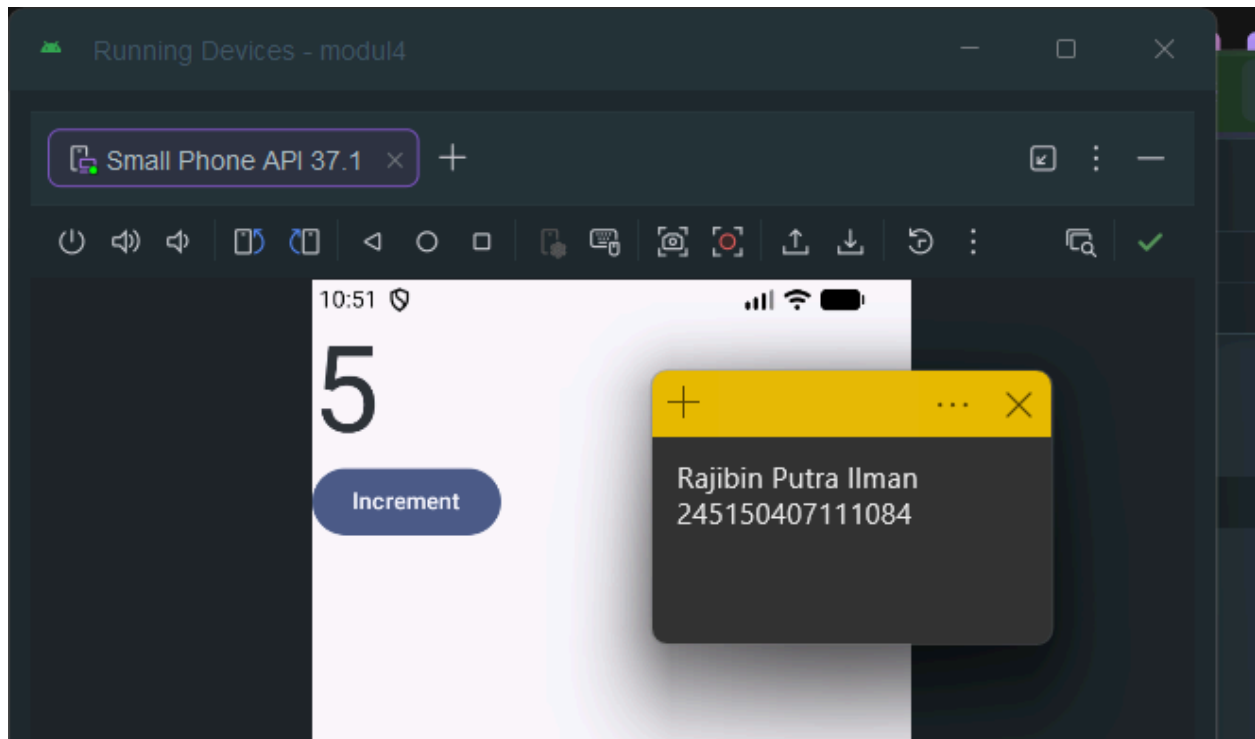
Munculnya Snackbar dengan pesan "Number 4 is shown!" yang dipicu oleh `LaunchedEffect(number)`.

12. Gambar 4.5.3 LaunchedEffect Percobaan 2 (Kondisi Awal):



Kondisi awal aplikasi pada Pra-Percobaan 2 saat pertama kali dijalankan dengan nilai awal 0.

13. Gambar 4.5.3 LaunchedEffect Percobaan 2 (Setelah Delay 2 Detik):



Hasil perbaruan angka menjadi acak (5) setelah dipicu oleh `LaunchedEffect(Unit)` yang mengambil data async melalui `Repo.getData()` selama 2 detik.